# How can Feature Selection reduce overfitting?

Last Updated : 27 Feb, 2024

The development of precise models is essential for predicted performance in the rapidly developing area of machine learning. The possibility of overfitting, in which a model picks up noise and oscillations unique to the training set in addition to the underlying patterns in the data, presents an inherent problem. As a potent weapon against overfitting, feature selection offers a way to improve the efficiency and generality of the model.

## How does Overfitting occur?

When a machine learning model learns all the intricacies and noise in the training data to the point where it adversely affects the model's performance on new data, this is known as overfitting. The reason for this is that the model is very complicated in comparison to the volume and noise level of the training data. Poor generalization can result from overfitting, which happens when a model is too adaptive and fits the training data exactly—including noise. In addition, the model may be able to memorize the training data rather than understand the underlying patterns if it has too many parameters in comparison to the number of observations in the training set.

## What is a Feature Selection?

Feature selection is the process of selecting a subset of relevant features (variables, predictors) for use in model construction. The goal of feature selection is to improve the model's performance by reducing overfitting, increasing model interpretability, and reducing computational complexity.

## How can Feature Selection reduce overfitting?

Feature selection can reduce overfitting by simplifying the model and focusing on the most relevant features. Here's how it helps:

1. **Reduced Complexity:** By selecting only the most important features, the model becomes simpler with fewer parameters. A simpler model is less likely to overfit because it has less capacity to memorize noise in the data.
2. **Prevents Memorization of Noise:** Overfitting occurs when a model learns the noise in the training data as if it were a real pattern. Feature selection helps remove noisy features, reducing the chances of overfitting by preventing the model from learning these spurious correlations.
3. **Better Generalization:** By focusing on the most relevant features, feature selection helps the model generalize better to unseen data. It ensures that the model learns the underlying patterns in the data rather than memorizing noise.
4. **Improved Model Performance:** Removing irrelevant or redundant features can improve the model's performance by allowing it to focus on the most important features, leading to more accurate predictions.
5. **Faster Training:** With fewer features, the model requires less time and computational resources to train. This can be especially beneficial when working with large datasets or complex models.

Overall, feature selection helps reduce overfitting by simplifying the model, preventing the memorization of noise, and focusing on the most relevant features, leading to better generalization and improved performance.

## Stepwise Guide to Select the Best Features

### Implementation on real life dataset

**Importing neccessary libraries**

Python3

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, SelectPercentile, RFE, SelectFromModel
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

**Loading and Splitting Dataset**

- The `X` variable contains the features (sepal length, sepal width, petal length, petal width) of the flowers, and the `y` variable contains the target labels (species of iris).
- The `train_test_split` function is used to split the data, where 70% of the data is used for training (`X_train`, `y_train`) and 30% is used for testing (`X_test`, `y_test`).
- The `random_state` parameter is set to 42 to ensure reproducibility of the split.

Python3

```
# Load the iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

**1. Correlation for feature selection**

Using correlation is a common technique to identify relationships between features in a dataset. It helps in understanding which features are closely related and can potentially be redundant. Here's how you can use correlation:

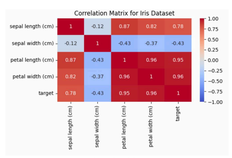
1. **Calculate Correlation Matrix:** Compute the correlation matrix of your dataset. This matrix shows the correlation coefficient between each pair of features.
2. **Visualize Correlation Matrix:** Create a heatmap of the correlation matrix to visualize the relationships between features. A higher correlation coefficient (closer to 1 or -1) indicates a stronger relationship.
3. **Identify Important Features:** Look for features that are highly correlated with the target variable (if it's a supervised learning problem) or with other features. Features with high correlation with the target are often good predictors, while features with high correlation with other features may be redundant.
4. **Select Features:** Based on the correlation matrix, you can decide to keep, drop, or further investigate certain features. High correlations between features may indicate multicollinearity, which can adversely affect some models (like linear regression).

Python3

```
# Calculate the correlation matrix
iris_corr_matrix = iris_df.corr()

# Create a heatmap of the correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(iris_corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Matrix for Iris Dataset')
plt.show()
```

**Output**:




*Correlation Matrix*

Here, we see that with target is mostly correlated with petal length and petal width, thus if we want to get the most important features we can select those too.

If you were to perform feature selection or feature importance analysis on this dataset, selecting petal length and petal width as the most important features could be a reasonable approach based on their high correlation with the target variable. These features provide valuable information for distinguishing between different species of iris.

**2. Filter Methods:**

Here's a brief definition of each of the filter methods:

1. **SelectKBest:** Selects the k highest-scoring features based on a scoring function (e.g., ANOVA F-value, chi-squared statistic) that measures the relevance of each feature to the target variable.
2. **SelectPercentile:** Selects the top features based on a percentile of the highest scores. For example, if you set the percentile to 10, it will select the top 10% of features.
3. **RFE (Recursive Feature Elimination):** Recursively removes the least important features and fits the model again until the desired number of features is reached. It uses the model's feature importance (or coefficient) to determine which features to eliminate.
4. **SelectFromModel:** Selects features based on a threshold of importance weights computed by an estimator (e.g., decision trees, linear models). It keeps only the features whose importance exceeds the threshold.
5. **Random Forest correlations:** Calculates the feature importances from a Random Forest model and uses them as a filter method. Features with higher importances are considered more important.

**A. SelectKBest:**

SelectKBest Selects the top k features based on a scoring function. In this case, k=2 is used to select the top 2 features.

Python3

```
# SelectKBest
selector_kbest = SelectKBest(k=2) # Select top 2 features
X_train_kbest = selector_kbest.fit_transform(X_train, y_train)
```

**B. SelectPercentile:**

Selects the top features based on a percentile of the highest scores. Here, `percentile=50` is used to select the top 50% of features.

Python3

```
# SelectPercentile
selector_percentile = SelectPercentile(percentile=50) # Select top 50% features
X_train_percentile = selector_percentile.fit_transform(X_train, y_train)
```

**C. RFE (Recursive Feature Elimination):**

Recursively removes the least important features until the desired number of features is reached. `n_features_to_select=2` is used to select the top 2 features.

Python3

```
# RFE (Recursive Feature Elimination)
estimator = LogisticRegression(max_iter=1000) # Base estimator
selector_rfe = RFE(estimator=estimator, n_features_to_select=2) # Select top 2 features
X_train_rfe = selector_rfe.fit_transform(X_train, y_train)
```

**D. SelectFromModel:**

Selects features based on a threshold of importance weights computed by an estimator. `threshold='mean'` is used to select features with importance greater than the mean importance.

Python3

```
# RFE (Recursive Feature Elimination)
estimator = LogisticRegression(max_iter=1000) # Base estimator
selector_rfe = RFE(estimator=estimator, n_features_to_select=2) # Select top 2 features
X_train_rfe = selector_rfe.fit_transform(X_train, y_train)
```

**E. Random Forest correlations:**

Uses feature importances from a Random Forest model as a filter method. Features with importance greater than `threshold=0.1` are selected.

Python3

```
# SelectFromModel
estimator = RandomForestClassifier() # Base estimator
selector_model = SelectFromModel(estimator=estimator, threshold='mean') # Select features with importance greater than mean
X_train_model = selector_model.fit_transform(X_train, y_train)

# Random Forest correlations
model = RandomForestClassifier()
model.fit(X_train, y_train)
importances = model.feature_importances_
threshold = 0.1
X_train_rf_corr = X_train[:, importances &gt; threshold]
```

**Comparing the selected features by different methods**

Python3

```
# Display selected features for each method
print(&quot;Selected features using SelectKBest:&quot;)
print(selector_kbest.get_support(indices=True))
print(&quot;\nSelected features using SelectPercentile:&quot;)
print(selector_percentile.get_support(indices=True))
print(&quot;\nSelected features using RFE:&quot;)
print(selector_rfe.get_support(indices=True))
print(&quot;\nSelected features using SelectFromModel:&quot;)
print(selector_model.get_support(indices=True))
print(&quot;\nSelected features using Random Forest correlations:&quot;)
print([i for i in range(len(importances)) if importances[i] &gt; threshold])
```

**Output:**

```
Selected features using SelectKBest:
[2 3]

Selected features using SelectPercentile:
[2 3]

Selected features using RFE:
[2 3]

Selected features using SelectFromModel:
[2 3]

Selected features using Random Forest correlations:
[2, 3]
```

The selected features using different methods are the same, which indicates that all methods agree on the importance of features 2 and 3. Features in the Iris dataset correspond to petal length and petal width, which are known to be highly informative for iris species classification.



**Next Article**

How to Perform Feature Selection for Regression Data

## Similar Reads

**Difference Between Feature Selection and Feature Extraction**

Machine learning models require input features that are relevant and important to predict the outcome. However, not all features are equally important for a prediction task, and some features might even introduce noise in the model. Featur...

8 min read

**Train Neural Networks With Noise to Reduce Overfitting**

Neural networks have revolutionized artificial intelligence but they often fall into the trap of overfitting which may potentially reduce the model's accuracy and reliability. To address this issue, we will be uncovering the noise-based...

7 min read

**Using Early Stopping to Reduce Overfitting in Neural Networks**

Overfitting is a common challenge in training neural networks. It occurs when a model learns to memorize the training data rather than generalize patterns from it, leading to poor performance on unseen data. While various regularization...

7 min read

**Can the number of epochs influence overfitting?**

Answer: Yes, an excessive number of epochs can contribute to overfitting in machine learning models.How Number of Epochs Influences Overfitting:Underfitting and Overfitting:Underfitting: Occurs when the model is too simple and fails t...

2 min read

**Is There Any Difference Between Feature Extraction and Feature Learning?**

Answer: Yes, feature extraction involves manually selecting characteristics from data, while feature learning allows models to automatically discover the features to be used for a task.Feature extraction and feature learning represent tw...

1 min read

**Parameters for Feature Selection**

Prerequisite : Introduction to Dimensionality Reduction Dimensionality reduction is the process of reducing the number of random variables under consideration, by obtaining a set of principal variables. It can be divided into feature selection...

5 min read

**ML | Extra Tree Classifier for Feature Selection**

Prerequisites: Decision Tree Classifier Extremely Randomized Trees Classifier(Extra Trees Classifier) is a type of ensemble learning technique which aggregates the results of multiple de-correlated decision trees collected in a "forest...

6 min read

**Feature Selection using Branch and Bound Algorithm**

Feature selection is a very important factor in Machine Learning. To get the algorithms to work properly and give near about perfect predictions, i.e to enhance the performance of a predictive model, feature selection is required. Large set o...

4 min read

**Feature Subset Selection Process**

Feature Selection is the most critical pre-processing activity in any machine learning process. It intends to select a subset of attributes or features that makes the most meaningful contribution to a machine learning activity. In order to...

12 min read

**SVM with Univariate Feature Selection in Scikit Learn**

Support Vector Machines (SVM) is a powerful machine learning algorithm used for classification and regression analysis. It is based on the idea of finding the optimal boundary between two classes that maximizes the margin between them....

10 min read

Article Tags : Machine Learning AI-ML-DS

Practice Tags : Machine Learning